



University of Oulu

Faculty of Electrical Engineering and Computer
Science

Fall Detection System Using Machine Learning, ESP8266, and GY-91 Sensor

Madhav Dahal (2409344) & Chathura Jayasekara (2406803)

Supervisor: Christian Schuss

A report submitted in fulfilment of the requirements of
the Wireless Management Project Course
Master's Programme in *Biomedical Engineering*

September 29, 2026

Abstract

This project aims to develop a wearable fall detection system utilizing a ESP8266 micro-controller and a GY-91 sensor. The system leverages machine learning to predict falls based on data collected from the sensors. A straightforward machine learning classification model is employed to identify patterns associated with falling and normal walking conditions. This report details the system's design, implementation, and evaluation, covering aspects such as data collection, preprocessing, model training, real-time prediction and notification via sms. Additionally, a demonstration video is included to showcase the prototype's functionality.

Contents

1	Introduction	1
2	Hardware and Software	1
2.1	Hardware:	1
2.2	Software:	1
3	Data Collection	1
3.1	Code of data collection using Arduino IDE	2
3.2	Python Code for Storing Data in a CSV File	3
4	Preprocessing	4
4.1	Process of Data Preprocessing	5
5	Machine Learning Model	5
5.1	Steps Involved in Model Training	6
6	Prediction	7
6.1	Real-time Implementation Step	8
6.2	Twilio SMS Alert	9
7	Video Demonstration	9
8	Results and Discussion	9
9	Conclusion	11

1 Introduction

Falls are associated with significant risk, particularly for the elderly, who are most vulnerable when alone at home or walking outdoors. A fall in such circumstances can quickly become a life-threatening emergency if help is not provided promptly. Especially when the incident goes unnoticed, it may take considerable time to reach the hospital, depending on the severity of the injury. In such situations, a fall detection and automated notification system can be a lifesaver, ensuring timely assistance and potentially preventing serious consequences. In addition, this can be extended to the use of cyclists, especially during harsh weather situations. This project develops a wearable fall detection system using an ESP8266 microcontroller, a GY-91 sensor, and machine learning algorithms. The system collects motion data, preprocesses it, trains a predictive model, and deploys it for real-time fall detection and SMS notification.

2 Hardware and Software

The system consists of the following components:

2.1 Hardware:

- ESP8266 Microcontroller
- GY-91 Sensor (Accelerometer & Gyroscope)
- Breadboard
- Jumper Wires
- Power supply through the USB cable.

2.2 Software:

- Python: Data preprocessing and training the machine learning model.
- Arduino IDE: To read the sensor data, we need to interface with the sensor and the plate number ESP8266.
- Twilio: Sending the SMS notification.

3 Data Collection

Data collection was carried out using the GY91 sensor, which integrates both an accelerometer and a gyroscope. Two distinct scenarios were considered: one involving

normal activities, and the other simulating fall conditions. Data from normal activities were recorded and labelled accordingly in a CSV file using Python. Similarly, fall scenario data were captured and labelled using the same Python-based approach. Both datasets were subsequently merged into a single CSV file for preprocessing and training of the machine learning model. The Python code used for each step is provided below.

3.1 Code of data collection using Arduino IDE

```
1 #include <Wire.h>
2 #include <ESP8266WiFi.h>
3 #include "MPU6050.h"
4 #include <Adafruit_BMP280.h> // Include the BMP280 sensor library
5
6 // MPU6050 and BMP280 I2C addresses
7 const int MPU_addr = 0x68;
8 MPU6050 mpu(MPU_addr);
9
10 // Variables for MPU6050 readings
11 int16_t accelX, accelY, accelZ;
12 int16_t gyroX, gyroY, gyroZ;
13
14 // Label for the data (0 = Normal, 1 = Fall)
15 int label = 1; // Change this manually or programmatically based on the
    scenario
16
17 void setup() {
18     Serial.begin(115200);
19     Wire.begin();
20
21     // Initialize MPU6050
22     Wire.beginTransmission(MPU_addr);
23     Wire.write(0x6B); // Power management register
24     Wire.write(0);    // Wake up the MPU6050
25     Wire.endTransmission(true);
26     Serial.println("MPU6050 Initialized");
27
28     // Print CSV Header
29     Serial.print("Accel_X,Accel_Y,Accel_Z,Gyro_X,Gyro_Y,Gyro_Z, Label");
30     Serial.println(); // Move to next line
31 }
32
33 void loop() {
34     // Read MPU6050 sensor values
35     mpu.getMotion6(&accelX, &accelY, &accelZ, &gyroX, &gyroY, &gyroZ);
36
37     // Convert acceleration and gyroscope values
38     float ax = accelX ; // Raw acceleration data
39     float ay = accelY ;
40     float az = accelZ ;
41     float gx = gyroX ; // Raw gyroscope data
42     float gy = gyroY ;
43     float gz = gyroZ ;
44
45     // Print CSV-formatted data
```

```

46     Serial.print(ax); Serial.print(",");
47     Serial.print(ay); Serial.print(",");
48     Serial.print(az); Serial.print(",");
49     Serial.print(gx); Serial.print(",");
50     Serial.print(gy); Serial.print(",");
51     Serial.print(gz); Serial.print(",");
52     Serial.println(label); // Add the label (0 or 1)
53     delay(100); // Adjust delay based on your sampling rate
54 }

```

Listing 1: Arduino Code for Data Collection from gy91 sensor

3.2 Python Code for Storing Data in a CSV File

Below is the Python code for storing data in a CSV file and combining two CSV files into one.

```

1  import serial
2  import csv
3
4  # Configure the serial connection (adjust the port and baud rate)
5  SERIAL_PORT = "COM3" # Replace with your ESP8266's serial port (e.g.,
    COM3 on Windows, /dev/ttyUSB0 on Linux)
6  BAUD_RATE = 115200
7
8  # Open the serial connection
9  ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
10
11 # Open a CSV file for writing
12 csv_file = open("falling.csv", "w", newline="")
13 csv_writer = csv.writer(csv_file)
14
15 # Write the header row
16 csv_writer.writerow(
17     ["Accel_X", "Accel_Y", "Accel_Z", "Gyro_X", "Gyro_Y", "Gyro_Z", "
    Label"])
18
19 try:
20     while True:
21         # Read a line from the serial monitor
22         line = ser.readline().decode('utf-8').strip()
23
24         # Skip empty lines
25         if not line:
26             continue
27
28         # Split the line into values
29         values = line.split(",")
30
31         # Write the values to the CSV file
32         csv_writer.writerow(values)
33         print(values) # Optional: Print to console for debugging
34
35 except KeyboardInterrupt:
36     print("Data collection stopped.")
37

```

```

38 finally:
39     # Close the serial connection and CSV file
40     ser.close()
41     csv_file.close()
42     print("Serial connection closed and CSV file saved.")

```

Listing 2: Python Code for Data Collection

The following code is used to combine two CSV files into one and clean the data by removing null entries and duplicate entries.

```

1  import pandas as pd
2  # # Use raw strings (prefix with r) or double backslashes to avoid
   escape character issues
3  normal_data = pd.read_csv(r"C:\Users\madha\Desktop\Python-Learning\
   Day-18\FALL DETection\walking.csv")
4  fall_data = pd.read_csv(r"C:\Users\madha\Desktop\Python-Learning\
   Day-18\FALL DETection\falling.csv")
5  #
6  #
7  # # Add a label column to each dataset
8  normal_data['Label'] = 0 # Label for normal activity
9  fall_data['Label'] = 1   # Label for fall
10 #
11 # # Combine the datasets
12 combined_data = pd.concat([normal_data, fall_data], ignore_index=
   True)
13 #
14 # # Save the combined dataset to a new CSV file
15 combined_data.to_csv('final_combined_data.csv', index=False)
16
17 # # Load the combined dataset
18 data = pd.read_csv('final_combined_data.csv')
19 #
20 # # Check the first few rows
21 print(data.head())
22 #
23 # # Check the distribution of labels
24 print(data['Label'].value_counts())
25 #
26 # # Check for missing values
27 print(data.isnull().sum())
28 #
29 # # Drop rows with missing values (if any)
30 data = data.dropna()

```

Listing 3: Python Code for Combining the CSV File and Data Cleaning

4 Preprocessing

The preprocessing phase includes loading the dataset, extracting features based on labels, normalizing the extracted features, and splitting the data into training and testing sets. The code for preprocessing is provided below.

```

1 import pandas as pd

```

```

2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4
5 # Load the combined dataset
6 data = pd.read_csv('final_combined_data.csv')
7
8 # Separate features and labels
9 X = data.drop(columns=['Label']) # Features
10 y = data['Label'] # Labels
11
12 # Normalize the features
13 scaler = StandardScaler()
14 X_scaled = scaler.fit_transform(X)
15 \begin{lstlisting}[language=Python, caption=Preprocessing Steps]
16 import pandas as pd
17 from sklearn.model_selection import train_test_split
18 from sklearn.preprocessing import StandardScaler
19
20 # Load the combined dataset
21 data = pd.read_csv('final_combined_data.csv')
22
23 # Separate features and labels
24 X = data.drop(columns=['Label']) # Features
25 y = data['Label'] # Labels
26
27 # Normalize the features
28 scaler = StandardScaler()
29 X_scaled = scaler.fit_transform(X)

```

Listing 4: Preprocessing Steps

4.1 Process of Data Preprocessing

- **Loading the Dataset:** The dataset is loaded from a CSV file.
- **Feature and Label Separation:** The features X are separated from the target labels y.
- **Normalization:** The features are scaled using the StandardScaler to ensure that all features have a mean of 0 and a standard deviation of 1.
- **Train-Test Split:** The dataset is split into training (80%) and testing (20%) sets with a fixed random seed for reproducibility.

5 Machine Learning Model

The training phase involves training a Random Forest Classifier, evaluating its performance, visualizing the results, and saving the trained model.

```

1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.metrics import accuracy_score, classification_report
3 import joblib

```

```

4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.metrics import confusion_matrix, roc_curve, auc
7
8 # Train the model
9 model = RandomForestClassifier(random_state=42)
10 model.fit(X_train, y_train)
11
12 # Evaluate the model
13 y_pred = model.predict(X_test)
14 print("Accuracy:", accuracy_score(y_test, y_pred))
15 print("Classification Report:\n", classification_report(y_test, y_pred)
16       )
17 # Confusion Matrix Visualization
18 cm = confusion_matrix(y_test, y_pred)
19 plt.figure(figsize=(6, 4))
20 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
21            xticklabels=['Normal', 'Fall'], yticklabels=['Normal', '
22            Fall'])
23 plt.xlabel('Predicted')
24 plt.ylabel('Actual')
25 plt.title('Confusion Matrix')
26 plt.show()
27 # ROC Curve Visualization
28 y_prob = model.predict_proba(X_test)[:, 1]
29 fpr, tpr, thresholds = roc_curve(y_test, y_prob)
30 roc_auc = auc(fpr, tpr)
31 plt.figure(figsize=(8, 6))
32 plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {roc_auc:.2f})', color='
33         blue')
34 plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
35 plt.xlabel('False Positive Rate')
36 plt.ylabel('True Positive Rate')
37 plt.title('ROC Curve')
38 plt.legend(loc='lower right')
39 plt.grid()
40 plt.show()
41 # Save the trained model
42 joblib.dump(model, 'fall_detection_model_final.pkl')
43 joblib.dump(scaler, 'scaler_final.pkl')
44 print("Model saved as 'fall_detection_final.pkl'")

```

Listing 5: Python Code for Model Training

5.1 Steps Involved in Model Training

- **Model Training:** A Random Forest Classifier is trained on the scaled training data.
- **Evaluation Metrics:** The model's performance is evaluated using accuracy and a detailed classification report.
- **Confusion Matrix:** A heatmap of the confusion matrix is plotted to visualize the model's predictions.

- **ROC Curve:** The Receiver Operating Characteristic (ROC) curve is plotted, and the Area Under the Curve (AUC) is calculated to evaluate the model's ability to distinguish between classes.

6 Prediction

The real-time fall detection system consists of the following components:

- **Sensor Data Collection:** Sensor data (accelerometer and gyroscope readings) is collected in real-time from an ESP8266 connected via a serial port.
- **Data Preprocessing:** The incoming sensor data is preprocessed and scaled using the same scaler used during training.
- **Prediction:** The preprocessed data is fed into the trained Random Forest Classifier to predict whether a fall has occurred.
- **SMS Alert:** If a fall is detected, an SMS alert is sent to a predefined phone number using the Twilio API.

The implementation involves loading the trained model, reading sensor data in real-time, making predictions, and sending SMS alerts. The code is provided below.

```

1 import serial
2 import pandas as pd
3 import joblib
4 import time
5 from twilio.rest import Client # For sending SMS using Twilio
6
7 # Load the trained model and scaler
8 model = joblib.load('fall_detection_model_final.pkl')
9 scaler = joblib.load('scaler_final.pkl')
10
11 # Define column names (must match training)
12 columns = ['Accel_X', 'Accel_Y', 'Accel_Z', 'Gyro_X', 'Gyro_Y', 'Gyro_Z',
13            '']
14
15 # Connect to ESP8266 (check your COM port)
16 ser = serial.Serial('COM3', 115200, timeout=1)
17 time.sleep(2) # wait for serial to initialize
18 # State variable to track fall detection
19 fall_detected = False
20
21 def send_sms_to_madhav():
22     account_sid = 'ACa7616be0f505d008412b9ff61b9b42ed'
23     auth_token = 'fa98710011959d5cf80c670ab75046db'
24     client = Client(account_sid, auth_token)
25     message = client.messages.create(
26         messaging_service_sid='MG7e3004aaf85af6cce70f10a6d8bf916c',
27         body='Fall Detected! Emergency',
28         to='+358417408270'
29     )

```

```

29     print(message.sid)
30 while True:
31     try:
32         line = ser.readline().decode('utf-8').strip()
33         if not line:
34             continue
35
36         values = line.split(',')
37         if len(values) != 6:
38             print("Invalid line:", line)
39             continue
40
41         float_values = [float(v) for v in values]
42         input_df = pd.DataFrame([float_values], columns=columns)
43         scaled_data = scaler.transform(input_df)
44         prediction = model.predict(scaled_data)
45         status = "FALL" if prediction[0] == 1 else "NORMAL"
46         print(f"Prediction: {status} | Raw: {float_values}")
47
48         # Handle fall detection logic
49         if status == "FALL":
50             if not fall_detected: # If a fall is detected for the
first time
51                 fall_detected = True
52                 print("Fall detected! Sending SMS...")
53                 send_sms_to_madhav() # Call the function to send an
SMS
54         else:
55             fall_detected = False # Reset the state when back to
normal
56
57     except Exception as e:
58         print("Error:", e)

```

Listing 6: Real-Time Fall Detection Implementation

6.1 Real-time Implementation Step

- **Loading the Model and Scaler:** The trained Random Forest Classifier fall-detection-model-final.pkl and the scaler scaler-final.pkl are loaded using the joblib library.
- **Connecting to ESP8266:** The system connects to the ESP8266 microcontroller via a serial port COM3 with a baud rate of 115200.
- **Reading Sensor Data:** Sensor data is read line by line from the serial port. Each line contains six comma-separated values representing accelerometer and gyroscope readings.
- **Preprocessing:** The raw sensor data is converted into a data frame and scaled using the same scaler used during training.
- **Making Predictions:** The scaled data is passed to the trained model for prediction. A prediction of '1' indicates a fall, while '0' indicates normal activity.

- **Handling Fall Detection:** If a fall is detected for the first time, the system sends an SMS alert using the Twilio API. The fall-detected flag ensures that duplicate alerts are not sent until the system returns to a normal state.

6.2 Twilio SMS Alert

The Twilio API is used to send SMS alerts. Below is a description of the key components:

- **Account SID and Auth Token:** These credentials authenticate the Twilio API requests.
- **Messaging Service SID:** This identifies the messaging service used to send the SMS.
- **Recipient Phone Number:** The phone number to which the SMS alert is sent is specified in the 'to' parameter.
- **Message Body:** The message body contains the alert text, e.g., "Fall Detected! Emergency".

7 Video Demonstration

A demonstration video showcasing the functionality of the fall detection system is available at the following link: [Demonstration Video 1](#). [Demonstration Video 2](#). [Demonstration Video 3](#). The videos demonstrate the entire workflow, including data collection, model deployment, and real-time fall detection.

8 Results and Discussion

In this study, we thoroughly evaluated how well the fall detection model performs by using key metrics and visual tools. The ROC (Receiver Operating Characteristic) curve helps us understand how the model balances between correctly detecting falls (true positives) and avoiding false alarms (false positives) at different thresholds. A curve that covers a large area, especially one with an AUC (Area Under the Curve) close to 1, shows that the model performs well in distinguishing between fall and non-fall events.

We also used a confusion matrix, which gives a clearer picture of the model's predictions by showing how many times it got things right or wrong — including true positives, true negatives, false positives, and false negatives. This helps us pinpoint where the model is performing well and where it might need improvement.

Lastly, we looked at the accuracy score, which tells us, in simple terms, how often the model made correct predictions overall. By combining all these tools, we can confidently

assess the model's strengths and weaknesses, making sure it's reliable and easy to interpret for real-time use.

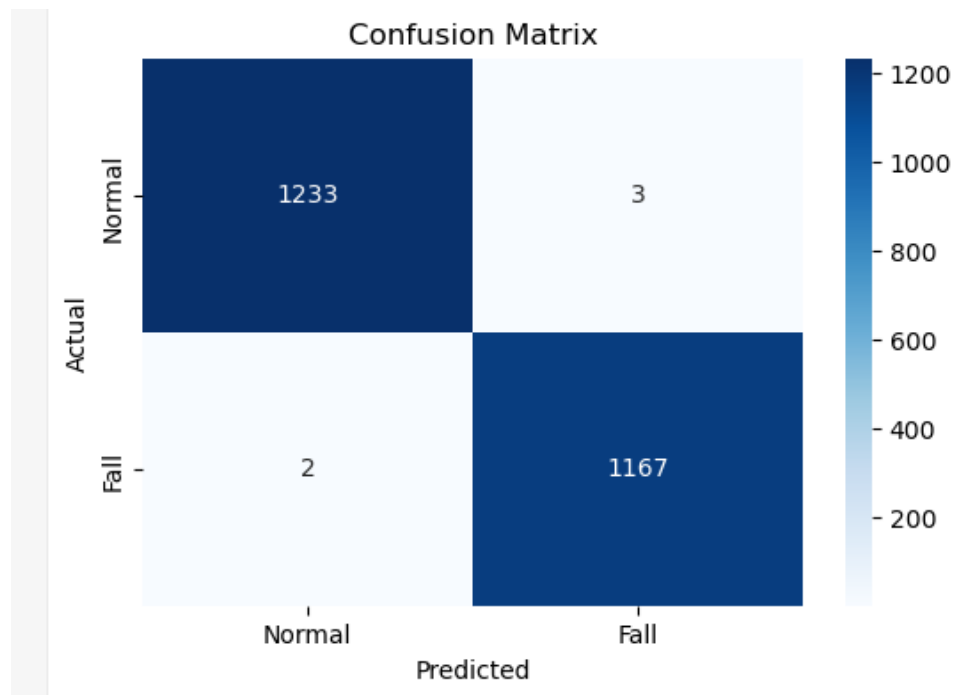


Figure 1: Confusion Matrix

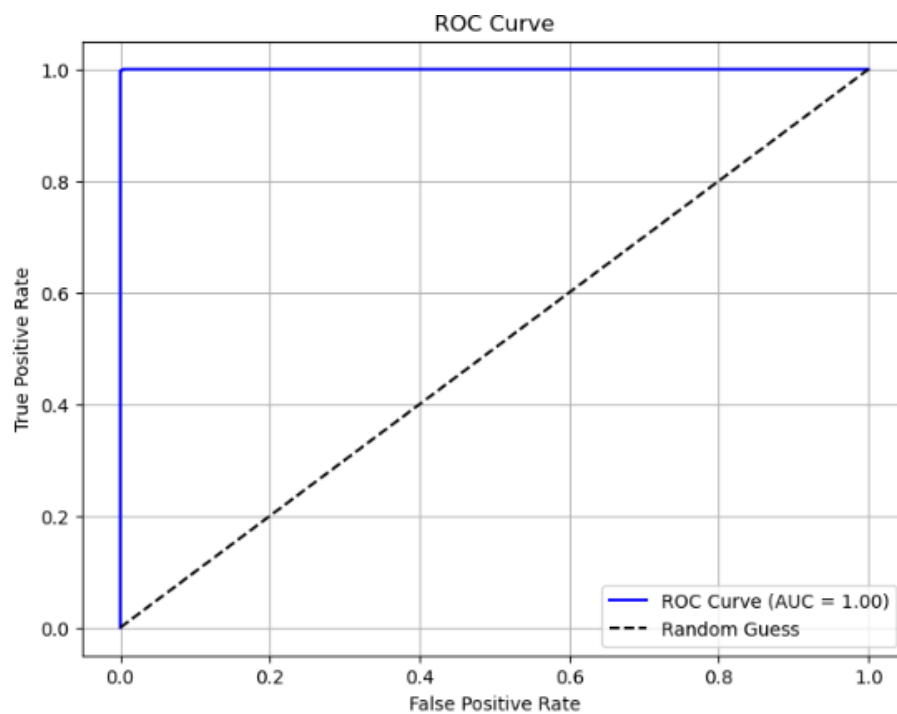


Figure 2: ROC Curve for Model Evaluation

```

Accuracy: 0.997920997920998

Classification Report:
              precision    recall  f1-score   support

     0           1.00       1.00       1.00       1236
     1           1.00       1.00       1.00       1169

 accuracy          1.00          1.00          1.00       2405
 macro avg          1.00          1.00          1.00       2405
weighted avg          1.00          1.00          1.00       2405

Confusion Matrix:
[[1233    3]
 [    2 1167]]

```

Figure 3: Accuracy of the model

9 Conclusion

This project successfully implemented a wearable fall detection system using an ESP8266, GY-91 sensor, and machine learning. Future work includes improving the model’s accuracy and integrating additional sensors to enhance overall system performance. An approach to increasing accuracy is by refining the feature extraction process, for example, by implementing a sliding window method to capture more meaningful and continuous patterns in the data.

References

- Twilio Documentation: <https://www.twilio.com/docs>
- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- Tang, D., Usman, A. B., Abba, A. (n.d.). Fall detection system with accelerometer and threshold-based algorithm. School of Computer Science, University of Sunderland; School of Architecture, Computing and Engineering, University of East London. <https://repository.uel.ac.uk/download/a32f7f3eacfffc303d8040c71353085af48ba1727646/Fall%20Detection%20System%20with%20Accelerometer%20and%20Threshold-based%20Algorithm.pdf>